PROJECT DEVELOPMENT PHASE

AUTO TICKET CLASSIFICATION USING FLOW DESIGNER

Full Project Documentation
Functional Requirements • Solution Workflow • Technical Architecture • Testing • Performance • Evidence

Date: 30 September 2026
Team ID: 6ab4d23f0fc666a751b534b0
Project Name: Auto Ticket Classification using Flow Designer
Maximum Member: 4

Team Leader:veeraprathap k

Member:karuppasamy m,mathavan k,muthukumar s

Table of Contents

1. 1. Project Overview
2. 2. Problem Statement
3. 3. Objectives
4. 4. Scope
5. 5. Solution Workflow
6. 6. Functional Configuration
7. 7. Functional Requirements
8. 8. Non-Functional Requirements
9. 9. Technical Architecture & Technology Stack
10. 10. Application Characteristics
11. 11. Classification Logic
12. 12. Data Model
13. 13. Exception Handling
14. 14. Testing Strategy
15. 15. Model / Classification Performance Test
16. 16. Performance Testing
17. 17. Validation Checklist
18. 18. Test Summary
19. 19. Evidence / Screenshots
20. 20. Implementation Procedure
21. 21. Risks and Mitigations
22. 22. Conclusion

1. Project Overview

The Auto Ticket Classification using Flow Designer project automates the classification of incoming support tickets. The workflow evaluates ticket information such as subject, description, category, priority indicators, and other configured fields, and assigns appropriate classification and routing values. The objective is to reduce manual triage, improve consistency, and accelerate ticket assignment.

The supplied project document defines the development-phase workflow as: ticket creation/update → Flow Designer trigger → ticket data collection → classification rules/decision logic → ticket update → queue/team routing → logging and performance monitoring.

2. Problem Statement

Support teams may receive a large number of tickets containing different issue types, priorities, and levels of detail. Manual review can create inconsistent classification and delays in assigning work. This project addresses that operational problem by applying configured Flow Designer conditions and decision logic automatically when eligible tickets are created or updated.

Source basis: the project file explicitly identifies automation of incoming ticket classification, reduction of manual triage, improved consistency, and faster assignment.

3. Objectives

- Automatically classify eligible support tickets.
- Evaluate subject, description, category, priority indicators, and configured fields.
- Apply documented classification rules and decision branches.
- Update classification, priority, and/or routing fields.
- Route tickets to the appropriate support queue or team.
- Provide fallback handling for unmatched or incomplete tickets.
- Capture execution logs and test evidence for validation.
- Measure classification accuracy, processing time, failure rate, and fallback rate.

4. Scope

In Scope	Description
Ticket trigger	Run when a new ticket is created or relevant ticket fields are updated.
Ticket data	Retrieve subject, description, category and other required fields.
Classification	Evaluate configured keywords, conditions, categories or rules.
Decision	Select the correct branch based on classification.
Update	Write classification, priority and/or routing values.
Routing	Send the ticket to the appropriate queue/team.
Exceptions	Handle missing data, unmatched tickets and flow errors.
Monitoring	Record execution and testing evidence.

Out of scope unless separately configured: a specific machine-learning model, third-party API integration, production infrastructure values, and measured performance numbers. The supplied documents leave these as configurable or to-be-entered fields.

5. Solution Workflow

23. Ticket is created or updated.
24. Flow Designer is triggered based on configured entry conditions.
25. Relevant ticket fields are collected and evaluated.
26. Classification rules/decision logic identify the appropriate ticket category.
27. The flow updates classification, priority or assignment fields.
28. The ticket is routed to the appropriate queue/team.
29. The result is logged for validation and performance monitoring.

Architecture flow:

User / Support Agent → Ticket Record → Flow Designer Trigger → Read Ticket Data → Classification Rules → Decision Branch → Update Ticket → Assignment/Routing → Execution Log / Monitoring

6. Functional Configuration

S.No.	Flow Component	Configuration / Purpose	Expected Result

1	Trigger	Run when a new ticket is created or relevant ticket fields are updated.	Flow starts only for eligible tickets.
2	Get/Read Ticket Data	Retrieve subject, description, category and other required fields.	Required ticket data is available.
3	Classification Logic	Evaluate configured keywords, conditions, categories or classification rules.	Correct classification is identified.
4	Decision	Branch the flow based on the classification result.	Ticket follows the correct decision path.
5	Update Ticket	Write classification, priority and/or routing values back to the ticket.	Ticket record is updated successfully.
6	Assignment/Routing	Route the ticket to the appropriate support queue or team.	Ticket reaches the intended team.
7	Exception Handling	Handle missing data, unmatched tickets and flow errors.	Exceptions are captured without breaking the process.

7. Functional Requirements

The supplied Solution Requirements template contains generic registration and confirmation requirements. For this project, the following ticket-classification functional requirements are derived directly from the provided Auto Ticket Classification workflow and test scenarios.

FR No.	Functional Requirement	Sub-Requirement / Story
FR-01	Ticket Trigger	Start the flow for eligible new or updated tickets.
FR-02	Ticket Data Retrieval	Read subject, description, category and required fields.
FR-03	Classification	Apply configured keywords, conditions, categories or rules.
FR-04	Decision Branching	Route the ticket through the branch matching the classification result.
FR-05	Ticket Update	Write classification, priority and/or routing values.
FR-06	Assignment	Route the ticket to the intended support queue/team.
FR-07	Fallback	Apply default category/queue for no-match cases.
FR-08	Exception Handling	Capture missing data and flow execution errors.
FR-09	Monitoring	Record execution logs and testing evidence.
FR-10	Performance Validation	Measure accuracy, execution time, volume, failure rate and fallback rate.

8. Non-Functional Requirements

NFR No.	Requirement	Project Interpretation / Validation
NFR-01	Usability	The classification flow should be understandable, consistent, and support repeatable ticket handling.
NFR-02	Security	Access to tickets, configuration, logs and administrative functions should be restricted to authorized users.
NFR-03	Reliability	Exceptions, missing data and unmatched tickets should be handled without breaking the process.
NFR-04	Performance	Measure average flow execution time and peak processing volume using execution logs/test data.
NFR-05	Availability	The workflow should be available in the configured execution environment; deployment details must be documented.
NFR-06	Scalability	The design should support increasing ticket volume by using efficient triggers, decision logic and appropriate platform capacity.

The supplied Solution Requirements template lists usability, security, reliability, performance, availability and scalability as the non-functional requirement categories.

9. Technical Architecture & Technology Stack

The supplied Technology Stack template requires an architectural diagram, application-logic blocks, infrastructure demarcation, external interfaces, data storage, and machine-learning interfaces where applicable. The project-specific supplied document does not specify a particular database, cloud provider, API, or ML model, so those items are marked as configurable/not specified rather than invented.

Component	Description	Technology / Configuration
User Interface	Support user/agent creates or updates a ticket.	Ticketing platform UI / configured application interface
Application Logic-1	Ticket trigger and field retrieval.	Flow Designer
Application Logic-2	Classification and decision logic.	Flow Designer conditions / rules
Application Logic-3	Ticket update and assignment.	Flow Designer actions
Database	Stores ticket records and configured fields.	Underlying ticketing-platform data store; exact DB not specified
Cloud Database	Optional cloud data service.	Not specified in supplied project document
File Storage	Evidence/screenshots/log files if stored externally.	Not specified
External API-1	Third-party integration, if required.	Not specified
External API-2	Third-party integration, if required.	Not specified
Machine Learning Model	Optional classifier if separately implemented.	Not specified; supplied workflow is rule/decision based

Infrastructure	Execution environment.	Platform/Flow Designer environment; exact local/cloud configuration not specified

Required architecture demarcation:

LOCAL / USER SIDE: Ticket entry and administration → APPLICATION / WORKFLOW: Flow Designer trigger, data retrieval, classification, decisions, updates and routing → DATA: Ticket records and execution logs → EXTERNAL SERVICES: only where configured → MONITORING: execution and performance evidence.

10. Application Characteristics

Characteristic	Design / Implementation	Technology
Open-Source Frameworks	No specific open-source framework is named in the supplied project documents.	Not specified
Security	Restrict access to ticket data, flow configuration and execution evidence; apply platform access controls.	Platform access control; exact controls not specified
Scalable Architecture	Flow is organized as trigger → processing → decision → update → routing, allowing modular rule maintenance.	Flow Designer
Availability	Use the configured workflow platform and monitor failed executions.	Platform workflow engine; deployment details not specified
Performance	Measure execution time, peak volume, failure rate and fallback rate using execution logs.	Flow execution logs / test dataset

The Technology Stack template specifically asks for security, scalable architecture, availability and performance considerations and examples such as IAM, encryption, load balancers and caching; no such specific technologies are asserted here because they are not specified in the project source.

11. Classification Logic

The classification mechanism in the supplied project is configured rule/decision logic. It evaluates keywords, conditions, categories or classification rules and then branches accordingly.

Input	Evaluation	Possible Output
Subject	Configured category keywords/conditions	Category
Description	Configured issue terms and conditions	Category / classification
Priority indicators	Urgent/critical or configured priority conditions	Priority
Existing classification fields	Relevant changes trigger re-evaluation when configured	Updated classification

No match	No configured category match	Default category/queue
Incomplete data	Missing/minimal description or required information	Fallback/unclassified
Conflicting indicators	Multiple category indicators	Configured precedence rule

12. Data Model

Field Group	Example Fields	Purpose
Ticket Identity	Ticket ID / record identifier	Identify the support ticket.
Content	Subject, Description	Provide classification input.
Classification	Category, Classification	Store the classification result.
Priority	Priority / priority indicators	Represent urgency.
Routing	Assignment group / queue / team	Direct work to the appropriate support team.
Status	Ticket status	Track processing state.
Audit / Monitoring	Execution result, timestamp, error details	Support validation and troubleshooting.

Exact field names and data types depend on the ticketing platform configuration and are not provided in the source.

13. Exception Handling

- Missing required data: capture the exception or use the fallback/unclassified path.
- Unmatched ticket: apply the configured default category/queue.
- Conflicting indicators: apply the documented precedence rule.
- Flow execution error: record the error and ensure the process does not break unexpectedly.
- Repeated updates: prevent unintended processing loops by configuring trigger conditions carefully.

14. Testing Strategy

Testing should validate normal classification, priority handling, fallback paths, conflicting indicators, update behavior, unmatched rules, execution errors and multiple-ticket processing.

TC ID	Test Scenario	Input / Condition	Expected Result
TC-01	New ticket with clear category indicators	Valid subject and description	Correct category assigned
TC-02	Ticket with high-priority indicators	Urgent/critical terms or configured priority condition	Priority set according to rules
TC-03	Ticket with incomplete description	Missing or minimal description	Fallback/unclassified path used
TC-04	Ticket with conflicting indicators	Multiple category keywords/conditions	Configured precedence rule applied
TC-05	Existing ticket updated	Relevant classification field changes	Flow re-evaluates only when configured conditions are met
TC-06	Ticket with no matching rule	No configured category match	Default category/queue applied

TC-07	Flow execution error	Simulated invalid/missing data	Error handled and recorded
TC-08	Multiple tickets processed	Batch of valid tickets	Tickets classified consistently without unacceptable delay

15. Model / Classification Performance Test

S.No.	Parameter	Value / Formula	Evidence
1	Total Test Tickets	[Enter count]	Attach screenshot/log
2	Correctly Classified Tickets	[Enter count]	Attach screenshot/log
3	Classification Accuracy	Correct / Total × 100	Attach report
4	Incorrectly Classified Tickets	[Enter count]	Attach screenshot/log
5	Unclassified / Fallback Tickets	[Enter count]	Attach screenshot/log
6	Average Processing Time	[Enter ms/sec]	Attach flow execution log
7	Flow Failure Rate	Failed Runs / Total Runs × 100	Attach monitoring evidence

Important: the supplied project file requires actual measurements to be entered from execution logs. No measured values were supplied, so this document does not fabricate them.

16. Performance Testing

Metric	Target / Benchmark	Observed	Status	Remarks
Classification accuracy	≥ [Target %]	[Enter]	[Pass/Fail]	Validate against labelled test tickets.
Average flow execution time	≤ [Target]	[Enter]	[Pass/Fail]	Measure from flow execution logs.
Peak processing volume	[Target tickets/min]	[Enter]	[Pass/Fail]	Execute a representative load.
Flow failure rate	≤ [Target %]	[Enter]	[Pass/Fail]	Review failed flow runs.
Fallback rate	≤ [Target %]	[Enter]	[Pass/Fail]	Investigate unmatched ticket patterns.

17. Validation Checklist

- All intended ticket fields are available to the flow.
- Trigger conditions prevent unnecessary executions.
- Classification rules are documented and tested.
- Decision branches cover normal, fallback and exception scenarios.
- Ticket updates are written to the correct fields.
- Routing/assignment is verified for each supported category.

- Duplicate or repeated updates do not create unintended processing loops.
- Execution logs and screenshots are captured as evidence.
- Performance results are recorded using the same test dataset and environment.

18. Test Summary

Item	Result
Total Test Cases	[Enter]
Passed	[Enter]
Failed	[Enter]
Overall Remarks	[Enter summary, defects and corrective actions]

19. Evidence / Screenshots

The supplied project document specifies five evidence screenshots. Insert the actual screenshots from the configured Flow Designer environment in the spaces below.

Screenshot 1 – Flow Designer trigger and start conditions.

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

Screenshot reference / filename: _____

Screenshot 2 – Classification decision/rule configuration.

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

Screenshot reference / filename: _____

Screenshot 3 – Ticket after classification and routing.

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

Screenshot reference / filename: _____

Screenshot 4 – Flow execution details / logs.

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

Screenshot reference / filename: _____

Screenshot 5 – Performance or test-result report.

[INSERT ACTUAL PROJECT SCREENSHOT HERE]

Screenshot reference / filename: _____

20. Implementation Procedure

30. Create or identify the ticket fields required for classification, priority and routing.
31. Configure the Flow Designer trigger for new tickets and/or relevant ticket updates.
32. Configure actions to retrieve subject, description, category and required fields.
33. Create classification conditions using the project's approved keywords, categories and business rules.
34. Create decision branches for supported classifications.
35. Configure ticket update actions for classification, priority and routing values.
36. Configure assignment to the correct queue/team.
37. Create fallback handling for incomplete or unmatched tickets.
38. Configure exception handling and execution logging.
39. Execute the eight supplied functional test scenarios.
40. Run the performance test using a defined labelled dataset and record actual measurements.
41. Capture the five required screenshots and complete the test summary.

21. Risks and Mitigations

Risk	Potential Effect	Mitigation
Ambiguous ticket text	Wrong classification	Use documented precedence rules and fallback handling.
Missing description	Unclassified ticket	Use fallback/unclassified path and capture missing-data evidence.
Rule overlap	Conflicting classifications	Define and test precedence rules.
Unnecessary triggers	Extra executions / possible loops	Restrict trigger conditions to relevant changes.
Flow errors	Ticket not routed	Capture errors, test exception paths, and monitor failed executions.
High ticket volume	Processing delay	Measure peak processing volume and execution time; optimize configuration if targets are not met.
Insufficient evidence	Difficulty validating results	Capture screenshots and execution logs for each required test area.

22. Conclusion

The Auto Ticket Classification using Flow Designer solution provides a repeatable and auditable mechanism for classifying and routing support tickets. The project workflow automates trigger handling, data evaluation, classification, decision branching, ticket updates, routing, exception handling and monitoring. Final acceptance should be based on

recorded functional results, classification accuracy, execution performance, exception handling, and evidence captured during testing.

Source note: This document consolidates and expands the three supplied project templates/documents while preserving their stated terminology and marking unspecified implementation values as configurable rather than inventing them.

Appendix A – Source Documents Used

Source	Use in this consolidated document
Auto_Ticket_Classification_using_Flow_Designer(1).docx	Primary project workflow, functional configuration, test scenarios, performance metrics, checklist and evidence requirements.
Solution Requirements.docx	Functional/non-functional requirement structure and requirement categories.
Technology Stack - Template.docx	Technical architecture, component/technology table, application characteristics and architecture documentation requirements.